

SISTEMAS DISTRIBUÍDOS E MOBILE • ATIVIDADE 01

Protocolos de Comunicação & Análise de APIs

Estudo do Protocolo SSH (Secure Shell) e Análise do Thunder
Client

Heitor Zeferino Siqueira:42521884

Rafael Kaique Mesquita Silva:42422386

Parte 1

Protocolo de Aplicação da Arquitetura TCP/IP:
SSH

Objetivo do Protocolo SSH

- ✓ **Acesso Remoto Seguro:** Protocolo da camada de aplicação projetado para gerenciamento e controle de computadores sobre redes não confiáveis.
- ✓ **Substituição de Legados:** Desenvolvido para substituir protocolos inseguros em texto puro, como Telnet, Rexec e rsh.
- ✓ **Porta Padrão TCP 22:** Opera via camada de transporte garantindo a entrega confiável de pacotes e tráfego totalmente encriptado.
- ✓ **Integridade dos Dados:** Garante que os comandos executados e dados trafegados não sejam interceptados (*sniffing*) ou adulterados.

Modo de Funcionamento do SSH



1. Handshake & Troca

O cliente conecta ao servidor (porta 22). Usando criptografia assimétrica, ambas as partes negociam e estabelecem uma chave simétrica de sessão.



2. Autenticação

O usuário se autentica com segurança utilizando senhas encriptadas ou pares de chaves criptográficas pública/privada (RSA, Ed25519).



3. Canal Criptografado

Toda a comunicação subsequente é protegida por criptografia simétrica e pacotes verificados com HMAC para integridade.

Exemplos Práticos de Utilização



Administração Remota de Servidores Acesso ao terminal de comandos de servidores Linux/Unix remotos na nuvem (AWS, Azure, DigitalOcean) para manutenção e suporte.



Transferência Segura de Arquivos (SCP / SFTP) Envio e download seguro de arquivos e atualizações de código diretamente pelo canal criptografado do SSH.



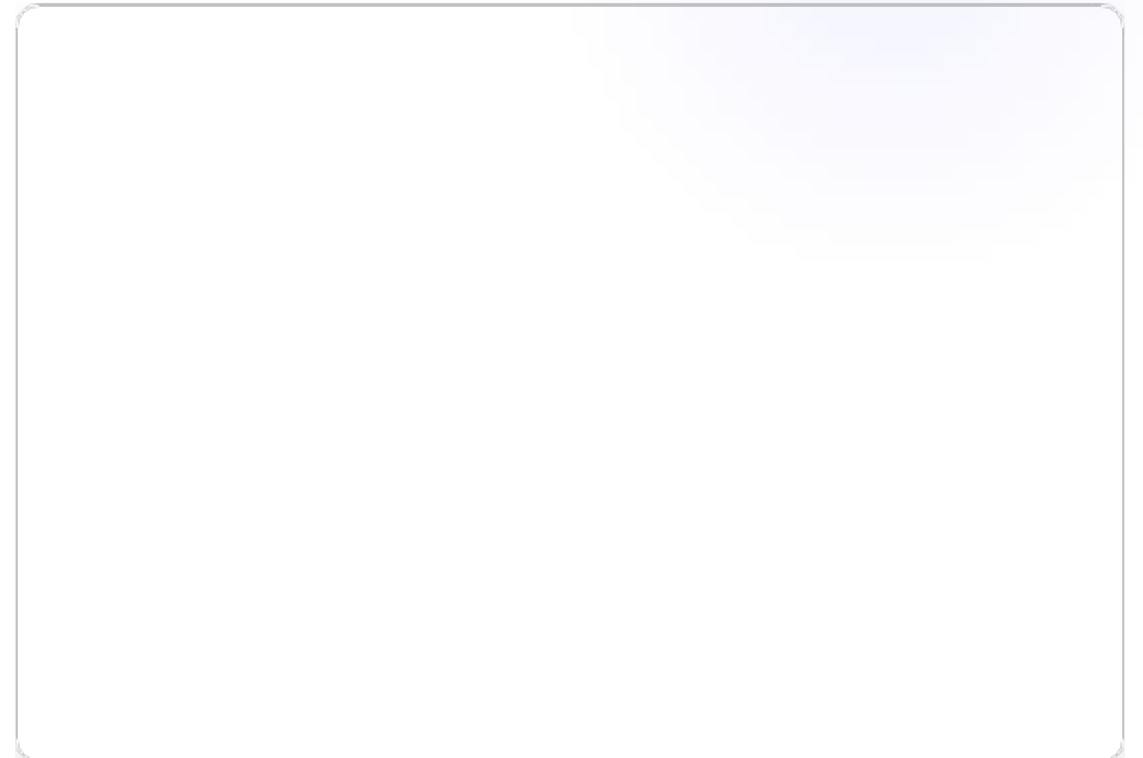
Tunelamento e Redirecionamento de Portas (Port Forwarding) Criação de túneis seguros para acessar bancos de dados e serviços internos de uma rede privada a partir da máquina local.

Parte 2

Análise de Ferramenta de API: Thunder
Client

Visão Geral: Thunder Client

- ✓ **Cliente HTTP Leve:** Extensão integrada nativamente ao Visual Studio Code para execução e testes de requisições de API (REST/GraphQL).
- ✓ **Foco em Produtividade:** Permite testar endpoints sem a necessidade de abrir softwares externos pesados.
- ✓ **Organização Eficiente:** Suporta a criação de coleções de requisições, testes automatizados básicos e gerenciamento de variáveis de ambiente.
- ✓ **Alternativa Moderna:** Uma solução ágil frente a ferramentas consolidadas do mercado como Postman e Insomnia.



Thunder Client: Pontos Positivos vs Negativos

✓ Pontos Positivos

- ✓ **Integração Direta no VS Code:** Baixo consumo de memória RAM e inicialização instantânea.
- ✓ **Simplicidade e Agilidade:** Interface limpa e intuitiva para testes rápidos de requisições GET, POST, etc.
- ✓ **Coleções & Ambientes:** Organização prática de parâmetros, headers e variáveis (Local/Prod).
- ✓ **Uso sem Conta Obrigatória:** Salva requisições localmente sem exigência de login na nuvem.

✗ Pontos Negativos

- ✗ **Recursos Pagos no Plano Pro:** Limitações na sincronização em nuvem e colaboração na versão grátis.
- ✗ **Menos Robusto que o Postman:** Falta suporte avançado a fluxos complexos de automação CI/CD.
- ✗ **Acoplado ao VS Code:** Não possui aplicativo *standalone* para uso fora do editor de código.
- ✗ **Scripting Limitado:** Menor flexibilidade para scripts pré e pós-requisição avançados.